

السؤال الاول

Comment: Ctrl + K, Ctrl + C

Uncomment: Ctrl + K, Ctrl + U

/ + Single line: Ctrl

السؤال الثاني

.Runtime Error: The code compiles successfully but crashes while running

.Example: Dividing a number by zero (int z = 5 / 0;)

.Logical Error: The code runs without crashing, but gives the wrong output

.Example: Writing x + y when you actually wanted to subtract x - y

السؤال الثالث

Following naming conventions makes the code readable, consistent, and easy for other developers to understand quickly. It's the standard way to write C

السؤال الرابع

.Value Types: Store the actual data directly in the Stack

Reference Types: Store a pointer in the Stack, but the actual data is saved in the .Heap

السؤال الخامس

Why: Because when you divide a smaller number (2) by a larger number (7), the .division result is 0, and the remainder is the smaller number itself (2)

السؤال السادس

(Logical AND): Uses short-circuiting. If the first condition is false, it stops && .and ignores the second

(Bitwise AND): Evaluates both sides regardless of the first condition's & .outcome

السؤال السابع

Answer: Because a double holds decimal values and an int does not. Explicit casting is required by C# to confirm that you are aware of and accept the .potential data loss (truncating the decimals)

السؤال الثامن

Exceptions: FormatException if the input is not a number (like letters), and OverflowException if the number is too large. Handling: You can handle this by .using a try-catch block, or by using int.TryParse() instead of Parse

السؤال التاسع

Value of x: 7

.Why: ++x makes x = 6 first. Then x++ uses the 6 and increments x again to 7

Part 2

:The LinkedIn Post

Understanding how .NET manages memory is crucial for writing efficient C# code. The difference between value types and reference types isn't just syntax, it's about memory allocation. Value types like structs or ints are allocated directly on the Stack. This makes them fast to access, and they are automatically popped off when out of scope. Reference types like classes and arrays are allocated on the Heap. In this case, the Stack only holds a reference to the actual data in the Heap. Memory in the Heap requires the Garbage Collector to clean up unused objects, which can effect .performance if you create too many of them

السؤال الثاني

Compiled languages: Translate the entire source code into machine code before .execution. They are generally faster

Interpreted languages: Translate and execute the code line by line at runtime. .They are slower but more flexible

C# execution: C# is a hybrid. It is first compiled into Intermediate Language (IL).
Then, at runtime, the JIT (Just-In-Time) compiler translates the IL into native
.machine code

السؤال الثالث

.Implicit: Auto, safe (int -> double)

.Explicit: Manual (), data loss risk

.Parse: Strings only, crashes on null

.Convert: Any type, safe with nulls (returns 0)

البونص

:C# is Managed Code -5

It means the code runs inside the CLR (Common Language Runtime), which
.handles memory management automatically using the Garbage Collector

:Struct is like a Class -6

Because a struct can have variables, methods, and constructors exactly just like
a class. The only difference is that struct is a Value Type (Stack), but class is a
.Reference Type (Heap)